

# Red team testing for recommendation systems driven by large language models

Sbaaoui Idriss

Student ID: 22060004

Department / Major: Computer Science

University: 华东理工大学

Supervisor: 张恒润

**Abstract**—Large language model (LLM) modules are now used in many recommendation pipelines for retrieval, reranking, and explanation generation. This adds flexibility, but it also opens a new risk: recommendation outputs can shift when retrieval data or prompt-facing text is manipulated. This proposal presents a planned study, *Red team testing for recommendation systems driven by large language models*, using Rag-Poison-Lab as the experiment platform. The study tracks how poisoning signals move from indexing and retrieval to reranking and final item exposure. The core question is simple: where does manipulation first take hold, and how far does it travel? To answer that, the plan runs controlled baseline-versus-attacked comparisons with repeatable data preparation, dual indexes, trace logs, and metric reporting that covers both recommendation quality and attack impact. The work stays at proposal stage and defines research questions, a technical plan, environment assumptions, budget and sustainability limits, and a 20-week schedule. The expected contribution is a practical red-team testing process for LLM-driven recommendation systems that can support later thesis work on robustness and reliability.

## I. RESEARCH BACKGROUND

Recommendation systems are moving from closed ranking pipelines to mixed systems that combine retrieval engines, deep ranking modules, and natural-language interfaces. In LLM-assisted settings, recommendation is no longer only a score-prediction task. It also depends on retrieval and language understanding. Users may receive recommendations through generated explanations or conversational ranking requests, and both rely on text-processing stages [1], [2].

This shift is why this study is needed. In a traditional setup, utility metrics may be the main quality signal. In an LLM-driven retrieval pipeline, that alone is not enough. Final top- $K$  results can change because candidate evidence changes, reranking context changes, or adversarial payload text appears at the wrong stage. For that reason, this proposal treats the recommender as a *pipeline under adversarial pressure*, not just a single model.

Red-team testing is a good response. Instead of assuming benign inputs, it defines attacker goals and limits first, then tests whether the system remains stable. In recommendation tasks, this allows planned evaluation of

targeted promotion, poisoned exposure, ranking instability, and explanation-level influence under controlled conditions.

In this proposal, I use Rag-Poison-Lab because it already supports the core controls needed for this work: deterministic preprocessing, attack-profile configuration, baseline/attacked indexing, recommendation serving, trace extraction, and report generation [3]. The goal is not to present a finished security framework. The goal is to use this platform as a practical base for a clear proposal-stage study.

The broader motivation is reliability in AI-based choice systems. Recommendation outputs influence what users watch, read, and buy. If adversarial text consistently pushes those outputs, the issue is both technical and practical. A movie can climb a list for the wrong reason. This is why the proposed work focuses on a controlled evaluation workflow with clear evidence instead of informal attack demos.

## A. Fundamentals of Deep Learning

Deep learning matters in this proposal because recommendation quality now depends on learned representations across multiple data types. User interactions, item metadata, and text fields are mapped into latent spaces where ranking decisions are made [4]. This is useful, but it also means that small input changes can travel through the system in ways that are hard to see directly.

Three fundamentals are especially important for the planned study. First, embedding space structure shapes ranking outcomes: small shifts in representation can change candidate neighborhoods and recommendation order. Second, deep systems are sensitive to distribution shift, so strong benign performance does not guarantee stable behavior under poisoned corpora. Third, in LLM-assisted recommenders, behavior is spread across retrieval, reranking, and explanation stages. So the analysis should check each stage.

These fundamentals directly shape the proposal design. The study aims to preserve utility under benign settings, measure quality drops under adversarial settings, and identify where perturbations become strongest along the pipeline. This suggests that evaluation method matters

as much as model choice. The contribution is mainly in evaluation method, not new model design: this project does not claim a new deep model, but proposes a controlled way to evaluate robustness behavior in an existing recommendation stack.

Another practical point is resource limits. Retrieval depth, reranking mode, and batch scope affect both results and cost. The proposed workflow uses fixed experiment grids and clear run snapshots so that observed differences come from attack conditions, not hidden setting changes.

Finally, this section treats deep learning as something to analyze, not a black box. The study will interpret metric shifts with trace evidence whenever possible. This keeps the final thesis argument clearer and more credible.

### B. Neural Network

Neural networks provide core modeling tools for both recommendation ranking and language-based reasoning. Feed-forward layers, embedding modules, and transformer-style architectures map mixed inputs into ranking signals. Prior neural collaborative filtering work showed why this matters: user-item interactions can be modeled beyond linear assumptions, improving expressive power [5].

In this proposal, neural-network relevance appears in four practical aspects:

- 1) **Representation learning:** user preference context and item semantics are represented in latent spaces where nearest-neighbor and ranking decisions are computed.
- 2) **Nonlinear ranking behavior:** small input shifts can produce nontrivial output reordering, especially in reranking steps.
- 3) **Prompt-conditioned reasoning:** LLM reranking and explanation are generation processes sensitive to context framing.
- 4) **Compositional vulnerability:** vulnerabilities can emerge at inter-module boundaries rather than from a single neural block.

A key assumption in this proposal is that capability and robustness should be evaluated separately. A network can be expressive but still vulnerable under adversarially modified retrieval inputs. For that reason, I prioritize controlled baseline-attacked comparison over clean-data accuracy alone.

The project further separates *model-internal* and *pipeline-external* causes of ranking change. Model-internal causes refer to learned scoring behavior under stable inputs. Pipeline-external causes include manipulated retrieval records and prompt-facing payload fields that alter candidate pools before final scoring. This distinction is needed if the final analysis is expected to support practical security recommendations.

In addition, the thesis will analyze explanation outputs as part of neural behavior. Explanations are not only interface text. They also influence trust and perceived



Fig. 1. Planned overall architecture of the Rag-Poison-Lab red-team evaluation platform.

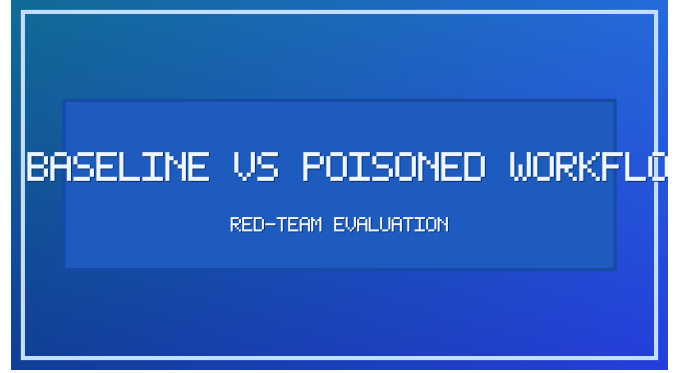


Fig. 2. Planned baseline-versus-poisoned workflow for controlled adversarial comparison.

relevance. If poisoned evidence changes explanation generation, the risk extends from ranking quality to user persuasion. The study treats explanation quality and consistency as an additional robustness concern.

### C. Retrieval-Augmented Generation and Ranking Pipeline

The core of this proposal is a retrieval-augmented recommendation workflow with explicit baseline and attacked branches. This choice is deliberate: the research question is about how adversarial influence moves through a full pipeline, not just one model component. Based on local audit, Rag-Poison-Lab already provides the needed setup—deterministic preprocessing, dual-index management, retrieval and ranking control, trace export, and evaluation scripts [3]. So the platform fit is practical, not theoretical.

Figures 1 and 2 summarize the planned experiment setup. In the thesis workflow, this architecture will be treated as a controlled plan: same user/query conditions, same output interface, and different index conditions.

Retrieval is not treated as only a preprocessing step in this proposal. It is an active factor in robustness. If ranking is the visible output, retrieval is the gate that decides what can be seen. Candidate generation relies on BM25-style search over title, genre, and synopsis fields [6]; poisoning in

TABLE I  
PLANNED PIPELINE STAGES AND OBSERVABLE ARTIFACTS

| Stage               | Planned observable outputs                                           |
|---------------------|----------------------------------------------------------------------|
| Data preparation    | Processed movie/rating/split/profile files; deterministic hashes     |
| Attack preparation  | Attack configuration snapshots; poisoned bulk metadata               |
| Indexing            | Baseline index and attacked index with aligned mappings              |
| Retrieval           | Retrieval query text; candidate list; poison marker visibility       |
| Ranking / reranking | Top- $K$ list, score signals, rerank prompt/response traces          |
| Explanation path    | One-sentence explanations per item with mode context                 |
| Evaluation          | Utility metrics, attack metrics, per-user deltas, aggregated reports |

those fields can change the candidate pool before ranking starts.

The proposed pipeline design includes two controls:

- **Mode control:** identical user/query settings are evaluated across baseline and attacked indexes.
- **Trace control:** retrieval evidence and reranking artifacts are recorded to support later analysis of where changes came from.

That setup lets the study ask not only *whether* outputs change, but also *where* and *why* the change appears along the pipeline. That distinction is central to this proposal.

#### *Planned Data and Index Rules*

To keep adversarial effects easy to interpret, the proposal introduces clear data and index rules. First, baseline and attacked indexes will share schema, mapping design, and retrieval settings; this avoids mixed effects from structure differences. Second, each attacked-index build will include metadata snapshots (attack type, poison fraction, source hash, output hash) to keep source tracking and repeatability.

The study also proposes deterministic data processing from raw interactions to processed artifacts. This matters because adversarial effects should be compared against a stable baseline. If split logic or preprocessing settings change between runs, baseline-attacked differences become difficult to interpret. Fixed preprocessing parameters and explicit run labels are part of the planned protocol.

For retrieval rules, query formulation policy will be fixed in paired baseline-attacked comparisons. User-context construction (top genres and liked titles), retrieval depth, and seen-item filtering will stay constant—unless a specific follow-up test targets one of these factors. This isolates attack effects from query-generation variance.

Finally, trace logging is treated as mandatory. For each evaluated mode, logs will keep retrieval query text, retrieved-document summaries, poison-marker visibility, and rerank context payloads when applicable. This keeps interpretation tied to evidence instead of score-only comparison.

## II. RELATED WORK

This proposal draws from four literature strands: neural recommendation, recommender poisoning, retrieval-augmented generation security, and LLM-based ranking.

### *A. Recommendation and Neural Ranking Foundations*

Pairwise ranking and neural recommendation literature established strong baselines for recommendation with implicit feedback [7], [5]. These works remain the right utility reference point for this thesis because robustness evaluation only matters if benign ranking quality stays acceptable.

### *B. Poisoning and Adversarial Manipulation in Recommenders*

Adversarial research has shown that recommendation behavior can be manipulated through injected or crafted data. Poisoning attacks on factorization-based collaborative filtering demonstrated both targeted attacks and broad quality drops [8]. Graph-based recommender poisoning further showed that attack effectiveness can remain high even with structural constraints [9]. Together, these findings support clear attacker modeling and limited perturbation budgets in this proposal.

This proposal extends that perspective to retrieval-augmented LLM recommendation settings. Instead of attacking interaction data alone, the planned experiments focus on manipulations in retrieval corpora and prompt-facing fields, where text artifacts can influence both ranking and generated explanations. That is the gap this thesis targets.

### *C. RAG and LLM Security Literature*

RAG systems combine external retrieval with generation to improve context use and factual grounding [1]. The same design also introduces corpus-integrity risks. Recent work reports vulnerability to retrieval poisoning and indirect prompt injection, where malicious external text alters downstream model behavior [10], [11], [12]. This risk maps directly to recommendation pipelines that expose textual item fields to reranking models.

The planned study differs from general RAG security evaluation by focusing on recommendation outcomes and exposure effects. The central question is not answer correctness. It is ranking stability under adversarial retrieval conditions.

### *D. LLMs as Rankers and Recommender Components*

Emerging evidence suggests that LLMs can function as zero-shot rankers for recommendation tasks [2]. This is attractive because it reduces dependence on task-specific fine-tuning, but it also implies that ranking logic can be sensitive to subtle textual manipulation. For this reason, the proposal treats LLM reranking as a strong attack surface that should be tested under controlled red-team settings. Useful, but risky.

### E. Dataset and Evaluation Context

MovieLens remains a widely used benchmark family for recommender research, with strong reproducibility value and clear historical context [13]. For ranking metrics, established information-retrieval concepts (for example, NDCG and rank-sensitive relevance analysis) provide the baseline method [14]. This proposal keeps that combination intentionally: it grounds the study in familiar recommender and IR practice before adding poisoning-specific evaluation layers.

Table II shows the gap addressed by this proposal: prior work is strong either on recommendation poisoning or on RAG/LLM security, but fewer studies connect both in one reproducible recommendation-focused protocol. This thesis does not try to replace model-centered advances. It focuses on improving how hybrid retrieval-generation recommendation systems are measured.

### III. TECHNICAL ROUTE

The technical route is designed for reproducible adversarial experiments rather than one-off attack demonstrations. The guiding choice is simple: enforce baseline-attacked matching controls, structured logs, and fixed stages so each comparison stays interpretable. Clear controls first, then scale.

#### A. Research Questions and Hypotheses

This study proposes the following research questions (RQs):

**RQ1:** How strongly do retrieval-corpus poisoning strategies alter top- $K$  recommendation outputs in LLM-assisted recommendation pipelines?

**RQ2:** How does ranking mode (deterministic vs. LLM rerank) influence robustness under identical attack budgets?

**RQ3:** Which trace-level signals best explain utility-robustness trade-offs across baseline and attacked runs?

The corresponding working hypotheses are:

- **H1:** Attacked indexes will increase poisoned or target-item exposure relative to baseline.
- **H2:** LLM reranking will provide semantic flexibility but may amplify susceptibility to poisoned textual cues.
- **H3:** Trace visibility (retrieved evidence + rerank context) will reduce ambiguity in interpreting attack effects.

These hypotheses are intentionally framed to be testable within the current platform design. They define a research direction, not predetermined outcomes. Not predictions. They also determine the order of the experimental route that follows.

#### B. Threat Model and Experimental Factors

The project adopts a limited attacker model:

- Attacker can manipulate a controlled fraction of indexed item records.
- Attacker cannot directly alter held-out test relevance labels.

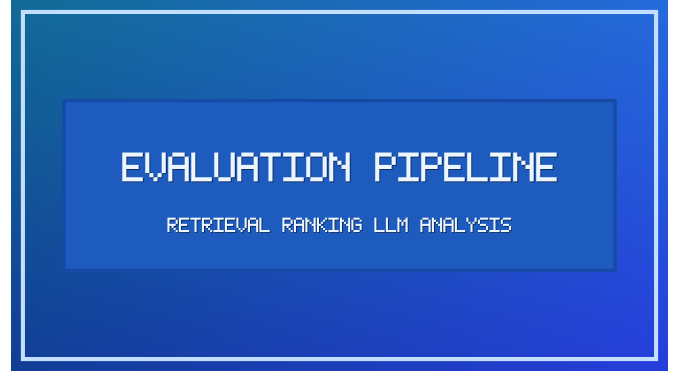


Fig. 3. Planned experiment flow from preprocessing and dual indexing to trace-driven robustness analysis.

- Attack channels include payload insertion and textual-field poisoning.
- Attack objective can be targeted promotion or untar-geted quality degradation.

Planned experimental factors include poison fraction, target selection policy, retrieval depth, ranking mode, and top- $K$  cutoff. Each run will store configuration and metadata snapshots for reproducibility and later audit. That way, when results diverge, the reason is easier to trace.

#### C. Execution Workflow

The planned workflow is:

- 1) Prepare deterministic dataset artifacts and user profiles.
- 2) Build baseline and attacked bulk files using fixed attack config.
- 3) Index both corpora under aligned schemas.
- 4) Run recommendation inference for selected user sets under both modes.
- 5) Collect traces (retrieval query, retrieved docs, rerank context).
- 6) Compute metric suite and aggregate deltas.
- 7) Generate reproducibility reports and summary artifacts.

This route keeps interpretation clear while keeping execution complexity realistic for a graduation timeline. Simple in wording, but demanding in practice.

#### Planned Experiment Matrix

To keep coverage broad but manageable, the proposal defines an experiment matrix that combines attack type, poison fraction, ranking mode, and evaluation scope. The matrix is organized to narrow key questions step by step:

- How does attack intensity affect utility degradation and target exposure?
- Does LLM reranking amplify or dampen attack effects under identical retrieval candidates?
- Which conditions produce the largest baseline-attacked divergence in top- $K$  recommendations?

TABLE II  
LITERATURE POSITIONING FOR THE PROPOSED STUDY

| Theme                            | Representative references | Main contribution trend                                                                                | Gap addressed by this proposal                                                            |
|----------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Neural recommendation baselines  | [7], [5]                  | Ranking quality optimization under implicit feedback and neural interaction modeling                   | Limited emphasis on retrieval-corpus adversarial channels in LLM-driven stacks            |
| Classical recommender poisoning  | [8], [9]                  | Demonstrates manipulability of recommendation outcomes through injected data or structure perturbation | Does not directly model RAG-style retrieval + LLM reranking pipelines                     |
| RAG and prompt security          | [1], [10], [11], [12]     | Shows retrieval dependency benefits and vulnerability to corpus/prompt attacks in LLM systems          | Typically centers QA/chat tasks rather than recommendation exposure and ranking stability |
| LLM ranking in recommenders      | [2]                       | Establishes LLM feasibility for ranking tasks with low-shot or zero-shot setup                         | Requires dedicated robustness evaluation under poisoning-aware recommendation experiments |
| Evaluation and benchmark context | [13], [14]                | Provides reproducible dataset baseline and rank-sensitive evaluation foundations                       | Needs integration with attack metrics and traceability in one unified protocol            |

Experiments are planned in stages: pilot matrix (small user set), core matrix (primary batch), and sensitivity matrix (focused follow-up tests). This step-by-step plan balances statistical coverage with compute limits.

#### Trace Analysis Protocol

The proposal treats trace analysis as a first-class method, not a debugging afterthought. For each paired baseline-attacked run, trace comparison follows three layers:

- 1) **Retrieval layer:** query text, candidate composition, and poisoned field visibility.
- 2) **Ranking layer:** score ordering changes, rerank candidate selection, and fallback behavior if applicable.
- 3) **Output layer:** final top- $K$  differences, explanation consistency, and target-item exposure movement.

This layered protocol is intended to answer a central question: does performance shift originate mainly in candidate retrieval, in reranking decisions, or in both. Representative trace cases will be reported for each attack profile to complement aggregate statistics. Numbers show what moved; traces help explain why.

#### Reproducibility and Run-Output Policy

The project proposes strict run-output management so each experiment can be reconstructed later. Planned output classes are:

- run-level metric JSON summaries;
- trace payloads for single-user deep-dive runs;
- attack and LLM configuration snapshots;
- markdown and CSV report outputs for comparison.

All outputs will be tagged by run label and timestamp. When possible, file hashes for core inputs (baseline and attacked bulk files) will be recorded for source checks.

Plainly, every major claim should map to a concrete run output.

#### D. Accuracy

Accuracy in this proposal is intentionally dual-objective: *recommendation utility* and *attack impact*. This split is important because a system can preserve utility while still becoming easy to manipulate. Utility metrics include Precision@K, Recall@K, HR@K, MRR@K, and NDCG@K [14]. Attack metrics include ASR@K, poisoned-item exposure rate, and baseline-attacked overlap shift. In short, good ranking alone is not enough.

Let  $R_u^K$  denote top- $K$  recommendations for user  $u$ , and let  $G_u$  denote relevant held-out items. The proposal uses:

$$\text{Precision@K}(u) = \frac{|R_u^K \cap G_u|}{K}, \quad (1)$$

$$\text{Recall@K}(u) = \frac{|R_u^K \cap G_u|}{|G_u|}, \quad (2)$$

$$\text{HR@K}(u) = \mathbb{I}(|R_u^K \cap G_u| > 0). \quad (3)$$

For rank sensitivity:

$$\text{NDCG@K}(u) = \frac{1}{\text{IDCG@K}(u)} \sum_{r=1}^K \frac{\text{rel}_{u,r}}{\log_2(r+1)}. \quad (4)$$

For attack success under targeted promotion:

$$\text{ASR@K} = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}(t \in R_u^K), \quad (5)$$

where  $t$  is the attack target item.

To characterize poisoning visibility in outputs, the proposal also defines:

$$\text{PER@K} = \frac{1}{|U|} \sum_{u \in U} \frac{|P \cap R_u^K|}{K}, \quad (6)$$

TABLE III  
PLANNED RED-TEAM EXPERIMENT MATRIX (PROPOSAL STAGE)

| Stage         | Attack type               | Poison fraction plan | Ranking mode               | User scope           | Main measurement objective                                            |
|---------------|---------------------------|----------------------|----------------------------|----------------------|-----------------------------------------------------------------------|
| Pilot A       | Prompt injection          | 0.01, 0.05           | Deterministic              | Single + small batch | Validate pipeline integrity and trace completeness                    |
| Pilot B       | Targeted promotion        | 0.01, 0.05           | Deterministic              | Small batch          | Verify ASR@K and target-rank-lift computation                         |
| Pilot C       | Prompt injection          | 0.05                 | LLM rerank                 | Small batch          | Compare rerank sensitivity to poisoned payload context                |
| Core 1        | Prompt injection          | 0.05, 0.10, 0.20     | Deterministic + LLM rerank | Batch mode           | Measure utility-robustness tradeoff under increasing attack intensity |
| Core 2        | Targeted promotion        | 0.05, 0.10, 0.20     | Deterministic + LLM rerank | Batch mode           | Measure targeted exposure behavior and ASR shifts                     |
| Core 3        | Untargeted degradation    | 0.05, 0.10, 0.20     | Deterministic + LLM rerank | Batch mode           | Measure broad relevance degradation and ranking instability           |
| Sensitivity 1 | Prompt injection          | fixed at 0.10        | Deterministic + LLM rerank | Batch mode           | Vary retrieval depth and inspect candidate contamination effects      |
| Sensitivity 2 | Targeted promotion        | fixed at 0.10        | Deterministic + LLM rerank | Batch mode           | Vary target policy and assess robustness consistency                  |
| Sensitivity 3 | Mixed profile stress test | fixed at 0.10        | Deterministic + LLM rerank | Full mode (limited)  | Study worst-case pipeline behavior under combined perturbations       |

TABLE IV  
PLANNED METRIC INTERPRETATION MAP

| Metric                 | Interpretation objective                                 |
|------------------------|----------------------------------------------------------|
| Precision@K, Recall@K  | Utility quality at recommendation cutoff                 |
| HR@K, MRR@K, NDCG@K    | Hit sensitivity and early-rank relevance behavior        |
| ASR@K                  | Position-sensitive relevance with logarithmic discount   |
| Poisoned Exposure Rate | Targeted attack effectiveness on exposure outcomes       |
| Overlap Shift          | Fraction of recommended items affected by poisoning      |
|                        | Stability gap between baseline and attacked ranking sets |

where  $P$  is the set of poisoned items.

The study will report per-user metrics, overall summaries, and baseline-attacked deltas. Confidence intervals will be estimated through repeated runs or bootstrap resampling when feasible. This reduces over-interpretation of single-run variation. Within this design, metric reporting is not only descriptive; it is part of the robustness argument.

#### E. Loss

Although the proposal emphasizes evaluation, loss formulations remain important for method grounding and future extension. This section links ranking objectives to robustness interpretation without turning the thesis into a model-training report. It keeps the method grounded while staying proposal-stage.

Pairwise ranking can be represented with BPR-style loss [7]:

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \log \sigma(\hat{y}_{u,i} - \hat{y}_{u,j}) + \lambda \|\Theta\|_2^2. \quad (7)$$

For LLM reranking components, direct differentiable training is not the immediate thesis goal. Instead, reranking is treated as an inference module whose robustness will be evaluated under controlled prompt and retrieval perturbations. A conceptual robust objective for later work can be written as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{utility}} + \alpha \mathcal{L}_{\text{attack}} + \beta \mathcal{L}_{\text{stability}}. \quad (8)$$

Here,  $\mathcal{L}_{\text{attack}}$  may penalize target overexposure under adversarial conditions, while  $\mathcal{L}_{\text{stability}}$  may constrain large ranking divergence between similar benign and perturbed contexts. This proposal does not claim these objectives are already implemented. They are presented as possible extensions informed by findings from the proposed experiments.

In the final thesis plan, this loss discussion serves two purposes:

- explain how existing ranking behavior can be interpreted in objective terms;
- motivate future robust-learning directions based on red-team findings.

#### F. Environmental Sustainability and Project Budgets

The project proposes a modest, sustainability-aware research strategy. Green AI arguments emphasize that computational cost should be reported and reduced where possible [15]. In this thesis context, that principle is applied through limited experiment grids, cached outputs,

TABLE V  
PLANNED RESEARCH BUDGET AND SUSTAINABILITY CONTROLS

| Item                       | Planned usage and control                                                     | Estimated cost (CNY) |
|----------------------------|-------------------------------------------------------------------------------|----------------------|
| Local compute / lab server | Fixed experiment windows, no repeated full retraining, controlled test matrix | 1,500                |
| LLM API backup             | Small validation-only cloud calls when local model comparison is needed       | 1,200                |
| Storage and backup         | Versioned run outputs, config snapshots, trace logs, and reports              | 300                  |
| Documentation and outputs  | Proposal/thesis files, charts, and reproducibility packaging                  | 500                  |
| Total                      | Graduation-level budget with clear efficiency limits                          | 3,500                |

and local-first execution. Practical limits are part of the design, not an afterthought.

Planned efficiency controls:

- reuse prepared datasets and dual-index outputs across experiments;
- limit reranking calls to predefined user batches and top- $K$  settings;
- prioritize structured follow-up tests over exhaustive brute-force search;
- track runtime/cost notes per experimental phase.

The proposal also includes a *compute accountability note* for each major milestone. The note will summarize approximate runtime, hardware profile, and key parameter ranges so robustness claims can be read together with resource usage. This keeps the evaluation plan realistic for a graduation project.

### G. Experimental Environment

The proposed environment follows the audited Rag-Poison-Lab stack and prioritizes reproducibility, control, and visibility [3]. This section makes implementation assumptions clear before experiments begin. It also keeps scope realistic for the 20-week plan.

1) *Python and Dependency Environment*: Core research execution will use Python 3.12 with uv-managed environments and lock files. This supports deterministic dependency installation for API logic, data processing, and evaluation tooling. Planned dependency groups are intentionally compact:

- API and command flow: FastAPI, Typer, pydantic settings;
- data processing: pandas and pyarrow;
- retrieval/infra client: Elasticsearch and HTTP clients;
- testing and quality: pytest-based validation workflows.

TABLE VI  
PLANNED RUNTIME STACK FOR EXPERIMENTS

| Layer                | Planned role in the study                                            |
|----------------------|----------------------------------------------------------------------|
| FastAPI backend      | Expose user, recommendation, trace, and settings endpoints           |
| Elasticsearch        | Host baseline and attacked indices; execute BM25 candidate retrieval |
| Attack builder       | Generate poisoned bulk corpora from attack profiles                  |
| Ranking engine       | Deterministic ranking and optional LLM reranking mode                |
| Evaluation/reporting | Compute metric deltas; export JSON/Markdown/CSV run artifacts        |
| React dashboard      | Visual baseline-attacked comparison and trace inspection             |
| Docker Compose       | Multi-service setup for reproducible local/lab runtime               |

The project will keep runtime config, attack config, and run outputs explicitly separated. This makes replication and audit easier, especially when comparing multiple poison profiles.

2) *Development Tools / IDE*: Development and experiment operation are planned around script-first tooling with optional GUI support. Scripts stay the default path:

- CLI commands for data preparation, attack generation, indexing, evaluation, and reporting;
- IDE support (VS Code/PyCharm) for static analysis, refactoring, and debugging;
- web dashboard for side-by-side comparison of baseline versus attacked outputs and trace evidence.

The dashboard is positioned as an interpretation aid rather than the primary experiment driver. But quantitative conclusions will rely on scripted runs and exported outputs.

3) *Core Frameworks and Runtime Stack*: The planned runtime stack combines backend APIs, retrieval infrastructure, and optional model providers. Table VI summarizes the target environment. Each layer has a direct role in either control or observability.

The environment is intended to support both *batch evaluation* and *single-user trace deep dive*. Batch mode will capture robustness trends, while trace mode will support pathway analysis for selected attack outcomes. That dual mode is part of the proposal logic: breadth for evidence and depth for interpretation.

### H. Planned Statistical and Validity Strategy

To improve scientific quality, the project defines a basic validity checklist. Nothing fancy, just consistent controls:

- fixed random seeds where stochastic components exist;
- clear separation of train/test interactions for relevance definitions;
- explicit attack budget documentation per run;
- paired baseline-attacked comparisons over identical user sets;
- report of skipped/failed cases to avoid reporting only successful runs.

Threats to validity are considered in advance:

- **Internal validity:** hidden effects from inconsistent configs;
- **Construct validity:** mismatch between attack goals and chosen metrics;
- **External validity:** transfer limits from MovieLens-like data to other domains.

These controls will be integrated into the execution schedule and reviewed at each major milestone. The intention is to keep validity checks active during the study, not only at write-up time.

#### Planned Thesis Outputs

At completion, the thesis is expected to produce several concrete outputs. These deliverables are concrete and testable:

- a formal red-team evaluation protocol for LLM-driven recommendation pipelines;
- an experiment output set that supports baseline-attacked reproducibility;
- comparison of deterministic and LLM reranking robustness behavior;
- practical recommendations for safer retrieval and ranking deployment.

These outputs are framed as planned deliverables and will be validated through the step-by-step timeline in the next section. This keeps the final section on schedule practical rather than symbolic.

#### IV. SCHEDULE

The project is planned over 20 weeks, aligned with graduation proposal expectations. The schedule allocates separate windows for threat-model refinement, pilot validation, full experiments, interpretation, and writing, so method decisions can still be revised before final conclusions are drafted. That pacing is intentional.

The schedule is intentionally iterative. Pilot findings are expected to inform experiment scaling, and interpretation checkpoints are placed before final chapter drafting to reduce the risk of forcing conclusions after the fact.

#### V. REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] Y. Hou, C. Gao, X. Wang, R. Song, X. He, K. Zhang, and J. Wang, “Large language models are zero-shot rankers for recommender systems,” *arXiv preprint arXiv:2305.08845*, 2024, presented at ECIR 2024. [Online]. Available: <https://arxiv.org/abs/2305.08845>
- [3] Rag-Poison-Lab Contributors, “Rag-poison-lab repository,” GitHub repository, 2026, accessed: 2026-03-07. [Online]. Available: <https://github.com/6ba3i/Rag-Poison-Lab>
- [4] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. [Online]. Available: <https://doi.org/10.1038/nature14539>
- [5] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural collaborative filtering,” in *Proceedings of the 26th International Conference on World Wide Web*, 2017, pp. 173–182. [Online]. Available: <https://doi.org/10.1145/3038912.3052569>
- [6] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- [7] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “BPR: Bayesian personalized ranking from implicit feedback,” *arXiv preprint arXiv:1205.2618*, 2012, originally introduced at UAI 2009. [Online]. Available: <https://arxiv.org/abs/1205.2618>
- [8] B. Li, Y. Wang, A. Singh, and Y. Vorobeychik, “Data poisoning attacks on factorization-based collaborative filtering,” in *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, 2016. [Online]. Available: <https://arxiv.org/abs/1608.08182>
- [9] M. Fang, N. Z. Gong, and J. Jia, “Poisoning attacks to graph-based recommender systems,” in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 381–392. [Online]. Available: <https://arxiv.org/abs/1809.04127>
- [10] K. Greshake, M. Abdelnabi, S. Saxena, C. Endres, M. Abdallah, and G. Fritz, “More than you’ve asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models,” *arXiv preprint arXiv:2302.12173*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.12173>
- [11] J. Zou, A. J. Troester, T. August, G. Neubig et al., “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” *arXiv preprint arXiv:2402.07867*, 2024, to appear in USENIX Security 2025. [Online]. Available: <https://arxiv.org/abs/2402.07867>
- [12] S. Xue, M. Yahya, B. Zhao, S. Islam et al., “BadRAG: Identifying vulnerabilities in retrieval-augmented generation of large language models,” *arXiv preprint arXiv:2406.00083*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.00083>
- [13] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 19:1–19:19, 2015. [Online]. Available: <https://doi.org/10.1145/2827872>
- [14] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. [Online]. Available: <https://nlp.stanford.edu/IR-book/>
- [15] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020. [Online]. Available: <https://doi.org/10.1145/3381831>



TABLE VII  
PROPOSED 20-WEEK SCHEDULE

| Weeks | Phase                         | Planned tasks                                                                                                                        | Expected deliverables                              |
|-------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 1–2   | Scope and proposal baseline   | Refine research questions, attacker model boundaries, and metric candidates; finalize proposal bibliography and compliance structure | Approved proposal baseline and execution checklist |
| 3–4   | Platform preparation          | Verify deterministic preprocessing and output schema; define experiment naming and run snapshot conventions                          | Reproducibility guide document                     |
| 5–6   | Attack profile design         | Finalize targeted/untargeted profile grid, poison fractions, and target policy assumptions                                           | Attack scenario matrix and config set              |
| 7–8   | Retrieval/rerank setup        | Standardize trace fields for retrieval query, candidate evidence, rerank input/output context                                        | Trace setup package                                |
| 9–10  | Pilot experiments             | Run limited-scope pilot evaluations to validate metric pipelines and report templates                                                | Pilot metrics and plan-refinement memo             |
| 11–12 | Batch experiments I           | Execute baseline-attacked comparisons for core attack profiles and primary $K$ values                                                | Batch I metrics and intermediate analysis          |
| 13–14 | Batch experiments II          | Add sensitivity analysis (poison fraction/ranking mode/retrieval depth); compare robustness trends                                   | Batch II metrics and follow-up test tables         |
| 15–16 | Interpretation and validation | Consolidate utility-robustness findings, check validity assumptions, and refine visualization assets                                 | Results draft and validity report                  |
| 17–18 | Thesis writing                | Complete method, experiments, discussion, and limitations chapters in full draft form                                                | Full thesis draft for supervisor review            |
| 19    | Final revision                | Apply feedback, format cleanup, citation audit, and appendix/reproducibility output polishing                                        | Submission-ready final manuscript                  |
| 20    | Defense preparation           | Build and rehearse defense presentation; prepare concise experiment narrative and Q&A notes                                          | Defense slide deck and final archive               |

TABLE VIII  
MILESTONE RISK-CONTROL SUMMARY

| Risk point               | Planned control action                                                |
|--------------------------|-----------------------------------------------------------------------|
| Experiment drift         | Lock config snapshots and fixed run labels                            |
| Metric mismatch          | Validate utility and attack metrics in pilot stage                    |
| Compute overuse          | Enforce limited experiment grid and cached artifacts                  |
| Interpretation ambiguity | Use trace-level evidence in addition to aggregate scores              |
| Timeline compression     | Maintain step-by-step deliverables with mid-course checkpoint reviews |